Listes, navigation, thème

Séance 3 · réconciliation, `Key`, `InheritedWidget`

Point de situation

Séance 2 : widgets, contraintes, `setState`, formulaires.

Aujourd'hui : une liste de stock filtrable, une fiche ouverte par son URL, un thème complet avec mode sombre.

Deux mécanismes du framework : la **réconciliation** de l'arbre d'Elements, et la **propagation** d'une valeur par `InheritedWidget`.

Aujourd'hui

Horaire	Bloc
0:00 – 0:15	Rappels
0:15 – 1:00	Listes, `Key`, Slivers, navigation
1:00 – 2:00	TP 1 : recherche, liste, navigation
2:00 – 2:15	Pause
2:15 – 2:55	Thème, `ThemeExtension`, `InheritedWidget`
2:55 – 3:55	TP 2 : thème, mode sombre, propagation du stock
3:55 – 4:15	Débrief

Listes

Construire uniquement ce qui est visible

`ListView.builder` ne crée que les widgets visibles

```
ListView(children: [for (final p in products) ProductRow(p)])   // everything, at once

ListView.builder(                                               // on demand
  itemCount: products.length,
  itemBuilder: (context, index) => ProductRow(products[index]),
)

GridView.builder(gridDelegate: …, itemBuilder: …)               // as a grid
```

- `ListView(children:)` crée tous les widgets à chaque build du parent, mais seules les lignes visibles deviennent `Element` et `RenderObject`
- `ListView.builder` n'appelle `itemBuilder` que pour les lignes visibles, plus une marge de défilement

`ListView.builder` construit une vingtaine de lignes sur 10 000

```
final products = List.generate(10000, (i) => 'Product $i');
```

Mesure	`SingleChildScrollView + Column`	`ListView(children:)`	`ListView.builder`
objets widget créés	10 000	10 000	une vingtaine
lignes construites, `debugPrintRebuildDirtyWidgets`	10 000	une vingtaine	une vingtaine

Seule la `Column` dans un `SingleChildScrollView` construit toute la liste.

`itemExtent`, quand la hauteur est fixe, donne l'étendue exacte et place n'importe quelle ligne sans mettre en page les précédentes.

`.builder` pour toute liste de taille variable

`ListView(children:)` : seulement pour une collection courte et fixe.

À chaque build, Flutter décide quels `Element` garder

- Rappel de la séance 2 : à chaque build, les widgets sont recréés, les `Element` sont conservés
- Flutter compare les nouveaux widgets aux anciens : c'est la **réconciliation**
- L'`Element` porte le `State` : le garder conserve l'état (texte saisi, case cochée), le recréer le remet à zéro

La comparaison se fait position par position

Une liste de trois lignes, dont la première disparaît :

```
build n                          build n+1
Column                           Column
├── ProductRow('A')              ├── ProductRow('B')
├── ProductRow('B')              └── ProductRow('C')
└── ProductRow('C')
```

Flutter compare les enfants d'un même parent **dans l'ordre** : l'ancien enfant 0, `ProductRow('A')`, est rapproché du nouvel enfant 0, `ProductRow('B')`.

Pour chaque paire, Flutter garde l'`Element` existant en lui donnant le nouveau widget, ou le jette et en crée un neuf.

`canUpdate` compare le type et la clé

Le choix est fait par `Widget.canUpdate`, une méthode statique du framework appelée pour chaque paire :

```
static bool canUpdate(Widget oldWidget, Widget newWidget) {
  return oldWidget.runtimeType == newWidget.runtimeType &&
      oldWidget.key == newWidget.key;
}
```

Réponse	Conséquence
`true`	l'`Element` est réutilisé, son `State` survit, seule la configuration change
`false`	l'`Element` est jeté, un autre est créé, l'état repart de zéro

Sans clé, `key` vaut `null` des deux côtés : deux lignes du même type donnent toujours `true`.

Sans clé, l'état suit la position

Chaque ligne garde une case cochée dans son `State` :

```
class _ProductRowState extends State<ProductRow> {
  bool _checked = false;   // lives in the Element, not in the product
  …
}
```

Liste	ligne 0	ligne 1	ligne 2
avant	Pasta ☑	Rice ☐	Milk ☐
Pasta retiré, sans clé	**Rice ☑**	Milk ☐	

Sans clé, l'`Element` de la ligne 0 est réutilisé pour Rice : il garde la case cochée de Pasta.

Une clé fait suivre l'état à sa donnée

```
Column(children: [
  for (final p in products)
    ProductRow(key: ValueKey(p.barcode), product: p),   // identity, not position
])
```

Liste	ligne 0	ligne 1
Pasta retiré, avec `ValueKey`	Rice □	Milk □

- `Column`, `ListView(children:)` : Flutter retrouve l'ancien `Element` qui porte la clé et le **déplace**
- `ListView.builder` : l'ancien `Element` est jeté et un nouveau créé, l'état repart de zéro ; `findChildIndexCallback` permet de le déplacer
- `ValueKey(index)` ne protège de rien : l'index se décale dès qu'une ligne disparaît

Une `Key` est l'identité d'un nœud

La question à se poser : **qu'est-ce qui, dans mes données, ne change pas quand la liste change ?**

Contraintes et géométrie d'un Sliver

Boîte	Sliver
`BoxConstraints` : largeur et hauteur	`SliverConstraints` : étendue visible, restante, direction
`Size`	`SliverGeometry` : étendue peinte, consommée, cache

Même protocole qu'en séance 2 : les contraintes descendent, la géométrie remonte.

`ListView` est un `CustomScrollView` à un seul sliver.

Un seul `ScrollView` pour l'en-tête et la liste

```
CustomScrollView(
  slivers: [
    SliverAppBar.large(
      title: const Text('My stock'),
      pinned: true,                  // title stays once collapsed
      expandedHeight: 200,
    ),
    SliverList.builder(
      itemCount: products.length,
      itemBuilder: (context, i) => ProductRow(products[i]),
    ),
  ],
)
```

`SliverAppBar` lit le décalage de défilement du `CustomScrollView` qui le contient : l'en-tête se replie à mesure que la liste défile.

Démonstration uniquement. Aucun TP ne porte sur les Slivers.

Navigation

Pile d'écrans et routes

`pop` renvoie une valeur à `push`

```
final product = await Navigator.of(context).push<Product>(
  MaterialPageRoute(builder: (context) => const AddProductPage()),
);

Navigator.of(context).pop(product);     // from the add screen
```

- `push` empile un écran, `pop` le retire **en transmettant une valeur**, que `push` renvoie
- Retour nullable : revenir sans valider renvoie `null`, cas nominal
- Sur le web, cette pile ne modifie pas l'URL

Déclarer les routes avec `go_router`

```
GoRouter(
  routes: [
    GoRoute(
      path: '/',
      pageBuilder: (context, state) => MaterialPage(child: const StockPage()),
      routes: [
        GoRoute(
          path: 'product/:barcode',
          pageBuilder: (context, state) => MaterialPage(
            child: ProductDetailPage(barcode: state.pathParameters['barcode']!),
          ),
        ),
      ],
    ),
  ],
)
```

- Une table de routes, avec des paramètres nommés
- `errorBuilder` pour les adresses inconnues

L'URL devient la source de vérité

- Coller `/#/product/3017620422003` ouvre la fiche ; le bouton « Retour » du navigateur fonctionne sans code.
- **Les liens profonds** utilisent la même table côté Android, en séance 10.
- **Les redirections** sont déclaratives : `redirect` renvoie une route quand un jeton manque. La garde de route de la séance 7 tient en trois lignes.
- **`state.pathParameters` est une chaîne**, jamais un objet : un code-barres inconnu doit avoir un écran.
- **Le `#` est la stratégie par défaut du web** : ce qui le suit n'atteint pas le serveur. `usePathUrlStrategy()` le retire, au prix d'une réécriture côté serveur, en séance 9.

`go` écrit l'URL, `push` empile

Appel	Pile	Barre d'adresse	Renvoie
`context.go('/product/…')`	celle de la route : la liste reste dessous	change	rien
`context.push('/add')`	un écran de plus, au-dessus de l'actuel	inchangée	un `Future`, la valeur de `pop`

Une fiche s'ouvre par `go` : son adresse se partage et sert de lien profond en séance 10. `Navigator.push` empile aussi, sans toucher à l'URL.

Un écran qui rend une valeur s'ouvre par `push`

```dart
// calling screen
final product = await context.push<Product>('/add');
if (!mounted || product == null) return;

// in /add, on submit
context.pop(product);
```

- Le `Future` de `push` se termine au `pop`, avec sa valeur
- Après l'`await`, l'écran appelant a pu disparaître : `mounted` d'abord

Déclarer chaque route avec `pageBuilder` et `MaterialPage`

```
GoRoute(
  path: '/',
  pageBuilder: (context, state) => const MaterialPage(child: StockPage()),
)
```

- Avec `builder`, `go_router` choisit le type de page en cherchant un `MaterialApp` au-dessus de lui
- Avec `go_router 18` et Flutter 3.47.8, cette détection échoue : chaque route devient une `NoTransitionPage`, sans transition ni `Hero`

Trois widgets de geste

Widget	Usage
`InkWell`	zone tactile **avec** retour visuel Material
`GestureDetector`	tous les gestes, aucun retour visuel
`Dismissible`	balayage pour retirer un élément

```
Dismissible(
  key: ValueKey(product.barcode),
  onDismissed: (direction) => stock.removeProduct(product.barcode),
  background: const _DismissBackground(),
  child: ProductListTile(product: product),
)
```

`Dismissible` retire son enfant de l'arbre **avant** tout rebuild de la liste.

TP 1 : recherche, liste, navigation

60 minutes

```
git switch session-03-start
flutter pub get
flutter run -d chrome
```

Exercice 1 de `TP.md`. Les trois chemins de routes sont imposés.

Pause

15 minutes

Thème

Color roles, design tokens, propagation

Un color role porte un usage, pas une teinte

```
ThemeData(
  colorScheme: ColorScheme.fromSeed(seedColor: Color(0xFF3F7D58)),
  useMaterial3: true,
)
```

`fromSeed` calcule une palette complète de **color roles** : `primary`, `onPrimary`, `surface`, `onSurfaceVariant`, `error`, `outlineVariant`…

`onSurfaceVariant` signifie « texte secondaire posé sur une surface », et sa valeur diffère en clair et en sombre.

Un mode sombre sans couleur en dur

```
MaterialApp(
  theme: ThemeData(colorScheme: ColorScheme.fromSeed(seedColor: seed)),
  darkTheme: ThemeData(
    colorScheme: ColorScheme.fromSeed(seedColor: seed, brightness: Brightness.dark),
  ),
  themeMode: ThemeMode.system,   // default
)
```

- Deux thèmes construits depuis la **même graine**, avec deux `Brightness`
- Condition : aucun widget ne cite une couleur
- Un `Colors.grey` lisible en mode clair devient illisible en sombre

Un style de texte porte un rôle, comme une couleur

```
final theme = Theme.of(context);
final secondary = theme.colorScheme.onSurfaceVariant;

Text(product.displayName, style: theme.textTheme.titleMedium)
Text(product.brand!, style: theme.textTheme.bodyMedium?.copyWith(color: secondary))
```

- `TextTheme` déclare quinze rôles : cinq familles, de `display` à `label`, en trois tailles
- La police se change une fois : `ThemeData(fontFamily: 'Inter')`, fichier déclaré sous `fonts:` dans le `pubspec.yaml`
- Un `TextStyle(fontSize: 14)` écrit dans un widget contourne le thème, comme une couleur en dur

`ThemeExtension` pour les design tokens

```dart
class PantryTokens extends ThemeExtension<PantryTokens> {
  const PantryTokens({required this.expirySoon});
  final Color expirySoon;

  @override
  PantryTokens copyWith({Color? expirySoon}) =>
      PantryTokens(expirySoon: expirySoon ?? this.expirySoon);

  @override
  PantryTokens lerp(PantryTokens? other, double t) => other == null
      ? this
      : PantryTokens(expirySoon: Color.lerp(expirySoon, other.expirySoon, t)!);
}
```

- Déclarée par `ThemeData(extensions: [tokens])`, lue par `Theme.of(context).extension<PantryTokens>()`
- `lerp` sert au changement de thème

Un widget lit un color role

Les valeurs de couleur ne s'écrivent que dans le fichier de thème.

Un `InheritedWidget` expose une valeur à tout son sous-arbre

```dart
class UserScope extends InheritedWidget {
  const UserScope({super.key, required this.name, required super.child});
  final String name;

  static UserScope of(BuildContext context) =>
      context.dependOnInheritedWidgetOfExactType<UserScope>()!;

  @override
  bool updateShouldNotify(UserScope old) => name != old.name;
}

Text(UserScope.of(context).name)   // anywhere below UserScope
```

- La valeur descend sans passer de constructeur en constructeur ; `Theme.of` et `MediaQuery.of` reposent sur ce mécanisme
- `of(context)` inscrit l'appelant comme dépendant : il est reconstruit quand la valeur change
- Le widget est immuable : la valeur vit dans un `State` au-dessus de lui, qui le reconstruit

Exposer `of(context)`, cacher le widget

- Le framework garde son `InheritedWidget` privé, `_InheritedTheme`
- Il expose une méthode statique, `Theme.of(context)`
- Même convention pour une portée écrite à la main : le point d'entrée est `of(context)`, l'implémentation reste un détail

Une portée absente lève une assertion

- `MediaQuery.of` sans `MediaQuery` au-dessus échoue en nommant le widget manquant
- `MediaQuery.maybeOf` renvoie `null`, pour qui sait s'en passer
- `Theme.of` retombe sur un thème par défaut
- Une portée de données suit le modèle de `MediaQuery.of` : sans elle, un stock vide afficherait une liste vide sans rien signaler

`updateShouldNotify` décide des reconstructions

```dart
@override
bool updateShouldNotify(_InheritedStock oldWidget) =>
    !identical(oldWidget.stock.products, stock.products);
```

Implémentation	Effet
comparer les valeurs utiles	les dépendants se reconstruisent quand c'est nécessaire
`=> true`	tous les dépendants se reconstruisent à chaque build du parent
`=> false`	rien ne se reconstruit jamais

`identical` suppose que chaque opération construise une **nouvelle** liste : une liste modifiée en place garde la même référence.

`Hero` anime un widget d'un écran à l'autre

```
Hero(
  tag: 'photo-${product.barcode}',
  child: ProductImagePlaceholder(initial: product.initial, size: 48),
)
```

- Flutter interpole position et taille pendant la transition de route
- Deux `Hero` de même tag dans un même écran lèvent une exception : le tag identifie un objet, pas un rôle

Les widgets `Animated…` interpolent seuls

```
AnimatedContainer(
  duration: const Duration(milliseconds: 200),
  curve: Curves.easeOut,
  width: selected ? 44 : 36,
)
```

- Le widget interpole lui-même entre l'ancienne et la nouvelle valeur
- `AnimatedOpacity`, `AnimatedAlign`, `AnimatedPadding` suivent le même modèle
- `AnimationController`, `Tween`, `CustomPainter` : en pointeurs ; la séance 10 y revient par la performance

TP 2 : thème, mode sombre, propagation du stock

60 minutes en binôme

Exercice 2 de `TP.md`. Le tableau des tokens est normatif.

Priorité : le thème, puis la propagation du stock. Le `Hero` et la jauge peuvent se terminer en dehors de la séance.

Points de relecture

- `Dismissible` : clé sur le code-barres ; l'index change après un retrait, `UniqueKey()` recrée chaque `State` à chaque build
- Liste : `ListView.builder`, qui ne crée le widget d'une ligne qu'à son affichage
- Code-barres inconnu : un état « produit introuvable », jamais une exception
- Mode sombre : aucune couleur en dur, contrôlée sur les six premiers produits
- Table de routage : elle ne reçoit plus le stock, chaque écran le lit par `of(context)`
- `updateShouldNotify` : une liste modifiée en place garde sa référence, `identical` ne voit aucun changement

Erreurs fréquentes

Erreur	Effet
`ValueKey(index)` sur un `Dismissible`	« A dismissed Dismissible widget is still part of the tree », sauf sur la dernière ligne
`onDismissed` qui ne retire pas le produit du stock	la même exception
`GoRouter` créé dans `build`	chaque rebuild de la racine ramène à la route de départ
`firstWhere` sans `orElse` sur un code-barres inconnu	écran rouge, « Bad state: No element »
`extension<PantryTokens>()!` sur un thème qui ne la déclare pas	« Null check operator used on a null value »
`of(context)` dans `initState`	« dependOnInheritedWidgetOfExactType<…>() … was called before … initState() completed »

Séance 4

Réseau, asynchrone, sérialisation : les vraies données d'Open Food Facts.

La portée du TP 2 reconstruit tous ses dépendants à chaque changement du stock : un compteur dans l'AppBar se reconstruit autant que la liste. Séance 5 : la granularité des reconstructions.

Questions

Pointeurs du cours 1

Ressource	Adresse
`ListView`	`api.flutter.dev/flutter/widgets/ListView-class.html`
Défilement et Slivers	`docs.flutter.dev/ui/layout/scrolling/slivers`
`Key`	`api.flutter.dev/flutter/foundation/Key-class.html`
`Dismissible`	`api.flutter.dev/flutter/widgets/Dismissible-class.html`
Navigation	`docs.flutter.dev/ui/navigation`
`go_router`	`pub.dev/packages/go_router`
Liens profonds	`docs.flutter.dev/ui/navigation/deep-linking`
Dépôt du cours	`github.com/mduruisseau/pantry-2026` → `session-03-start`

Pointeurs du cours 2

Ressource	Adresse
Thèmes, cookbook	`docs.flutter.dev/cookbook/design/themes`
`ColorScheme.fromSeed`	`api.flutter.dev/flutter/material/ColorScheme/ColorScheme.fromSeed.html`
Color roles Material 3	`m3.material.io/styles/color/roles`
`ThemeExtension`	`api.flutter.dev/flutter/material/ThemeExtension-class.html`
`TextTheme`	`api.flutter.dev/flutter/material/TextTheme-class.html`
Polices	`docs.flutter.dev/cookbook/design/fonts`
`InheritedWidget`	`api.flutter.dev/flutter/widgets/InheritedWidget-class.html`
`Hero`	`api.flutter.dev/flutter/widgets/Hero-class.html`
Animations implicites	`docs.flutter.dev/ui/animations/implicit-animations`
`AnimationController`, `Tween`	`docs.flutter.dev/ui/animations/tutorial`
`CustomPainter`	`api.flutter.dev/flutter/rendering/CustomPainter-class.html`